

➤ Imperative commands (single line commands to create the objects)

⌘ **kubectrl run my-pod --image=nginx** [here *run* is used, not *create*]

⌘ **restartPolicy:** always

always: restart pod even if the container exits with exit code **0**

onFailure: restart pod only if container's exit code is non-zero

never: never restart the pod even if container stops.

⌘ **namespace:** default

it is *logical* grouping of a cluster to separate logically like dev, prod, ..etc.

You can set the resource allocation according to namespace as well.

⌘ **labels:** *run: my-pod*

⌘ **port:** not exposed

[port is never exposed from container though, means from Service you can route to any port of the pod you want; *port* only there for ease of querying pods according to port or to display the usable port of the container to user when they get *kubectrl describe pod <pod name>*]

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: my-pod
    name: my-pod
spec:
  containers:
  - image: nginx
    name: my-pod
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
```

(default configs of the command)

⌘ **kubectrl create deployment my-deploy --image=nginx** [here *create* is used]

⌘ **Deployment name:** my-deploy

⌘ **ReplicaSet:** my-deploy-*<hash>*

⌘ **Pod:** my-deploy-*<hash>*-*<random>*

⌘ **labels:** *app:my-deploy*

- These are the things kubectl create commands can create. pod cannot be created using kubectl create command

clusterrole	Create a cluster role
clusterrolebinding	Create a cluster role binding for a particular cluster role
configmap	Create a config map from a local file, directory or literal value
cronjob	Create a cron job with the specified name
deployment	Create a deployment with the specified name
ingress	Create an ingress with the specified name
job	Create a job with the specified name
namespace	Create a namespace with the specified name
poddisruptionbudget	Create a pod disruption budget with the specified name
priorityclass	Create a priority class with the specified name
quota	Create a quota with the specified name
role	Create a role with single rule
rolebinding	Create a role binding for a particular role or cluster role
secret	Create a secret using a specified subcommand
service	Create a service using a specified subcommand
serviceaccount	Create a service account with the specified name
token	Request a service account token

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: my-deployment
  name: my-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-deployment
  strategy: {}
  template:
    metadata:
      labels:
        app: my-deployment
    spec:
      containers:
      - image: nginx
        name: nginx
        resources: {}
status: {}
```

(default configs of the command)

labels are automatically added.

- **ReplicaSet** cannot be created directly using imperative way, because Deployment is there to create and manage replicaset.
- **kubectl expose** takes replication controller, service, deployment, or pod and creates a *Service* to expose that.

```
vagrant@controlplane:~$ kubectl get svc,pod
NAME                                TYPE                CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
service/kubernetes                  ClusterIP           10.96.0.1     <none>          443/TCP     36m

NAME    READY   STATUS    RESTARTS   AGE
pod/my-pod  1/1     Running   0           34s
vagrant@controlplane:~$ kubectl expose pod my-pod --port 80
service/my-pod exposed
vagrant@controlplane:~$ kubectl get svc,pod
NAME                                TYPE                CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
service/kubernetes                  ClusterIP           10.96.0.1     <none>          443/TCP     37m
service/my-pod                      ClusterIP           10.96.68.232  <none>          80/TCP      6s

NAME    READY   STATUS    RESTARTS   AGE
pod/my-pod  1/1     Running   0           76s
```

you can see here, It created a service named *my-pod*. Its of type *ClusterIP* which is default.

- If you give **--type=NodePort** flag, then it'll create a service of type NodePort

```
vagrant@controlplane:~$ kubectl expose pod my-pod --port 80 --type NodePort
service/my-pod exposed
vagrant@controlplane:~$ kubectl get svc,pod
NAME                                TYPE                CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
service/kubernetes                  ClusterIP           10.96.0.1     <none>          443/TCP     39m
service/my-pod                      NodePort            10.96.119.232 <none>          80:32530/TCP 3s
```

The collage shows several terminal windows with the following commands:

- `$ kubectl run --image=nginx nginx`
- `$ kubectl expose deployment nginx --port 80`
- `$ kubectl scale deployment nginx --replicas=5`
- `$ kubectl create -f nginx.yaml`
- `$ kubectl delete -f nginx.yaml`
- `$ kubectl create deployment --image=nginx nginx`
- `$ kubectl edit deployment nginx`
- `$ kubectl set image deployment nginx nginx=nginx:1.18`
- `$ kubectl replace -f nginx.yaml`

[these are the imperative commands]

create -f is imperative because if already the same config present it'll fail, where as **apply -f** will not fail.

➤ Important trick

• **kubectl create deployment my-deploy --image=nginx --dry-run=client -o yaml**

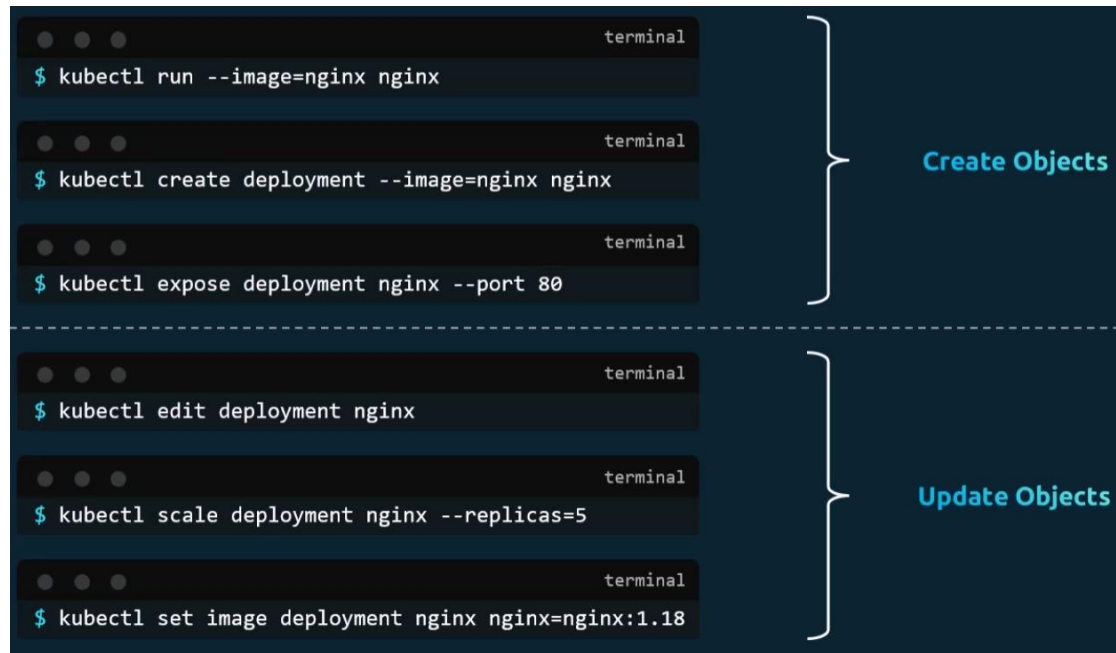
kubectl run my-pod --image=nginx --dry-run=client -o yaml

- it'll give the yaml file of default deployment, remove the unnecessary fields, customize that, and create deployment

- Because of the flag **--dry-run=client** it'll not create the deployment, just simulate and give the output as yaml.

➤ **kubectl get pod my-pod -o yaml**

- It'll give the running config of the pod as an yaml.
- Can be done with any types of objects i.e. pod, replica set, container runtime, deployment ..etc
- But it'll contain a lot of things, that needs to be removed.



➤ **edit vs replace command**

- kubectl edit** command open the running status of the object (pod, replicaset, replication controller, deployment, service ..etc) in a yaml file.

This configurations are there in the memory, not stored in the yaml file that was being used to create the object.

So, if you do **kubectl edit** then it'll not have any track of the changes. If someone later want to update the object, and he sees the yaml file present (not the running configuration yaml) and try to update, it might do some wrong changes in the running object.

- kubectl replace** command uses the physical yaml file to update the running object.

It does **dumb** update in the running object. Whatever there in the physical yaml, it'll update all the things and delete the configs which are not present in that physical yaml file.

By default it doesn't delete the current object (if anything doesn't match with physical yaml), to replace force fully you can give the flag **--force**.

```
vagrant@controlplane:~$ kubectl replace --force -f pd.yaml
pod "my-pod" deleted from default namespace
pod/my-pod replaced
```

It looks similar to *kubectl apply*, but *kubectl apply* does *intelligent* update. It finds the *diff* between running object and the physical yaml (used to update the object) and update those only.

It *doesn't delete any configs* which are not present in the current yaml but there in the running object.

kubectl apply is *declarative* command.

- If there are multiple files out of which objects will have to be created, then just give the path of the directory where those are present to *kubectl apply*, it'll create all those.

kubectl apply -f <path where all files are present>

--- kubectl apply internals ---

➤ [here I'll take an example of a deployment named *my-dep*]

➤ There are 3 things.

yaml (file that you write)

last applied config (when you execute *kubectl get deploy my-dep -o yaml*) [JSON format]

live config (when you execute *kubectl describe deploy my-dep*)

[*deploy* is shorthand of *deployment*]

⌘ Whenever you do *declarative* change using a *yaml* i.e.

kubectl apply -f my-dep.yaml

it changes both *live config*, *last applied config*. and obviously *yaml* is changed by you only.

⌘ Whenever you do *imperative* change using the command i.e.

kubectl scale deployment my-dep --replicas=5

it changes only *live config*. *Last applied config* will not be changed. And of course, *yaml* has not been changed.

command type	yaml	last applied config	live config
declarative	Y	Y	Y
imperative	N	N	Y

➤ There are a few commands [*deployment is used for example; can be used for any objects*]:

⌘ *kubectl diff -f <yaml file name>*

compare the *yaml with the live config* along with *last applied* and display how it'll look like after you execute *kubectl apply*.

⌘ *kubectl apply view-last-updated deployment <deployment name>*

show the last updated configs [which is present actually in JSON] in *yaml* format cleanly.

⌘ *kubectl get deployment <deployment name> -o yaml*

kubectl get -f <deployment yaml file name> -o yaml [it fetch from the file]

Both will display the *live configs of the deployment object*.

Also, there will be a JSON structure under *annotations* field where you can see *last updated config*.

➤ Stage-1: [create a deployment from yaml]

⌘ Initial yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ng-dep
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.16.1
        ports:
        - containerPort: 80
```

⌘ **kubectldiff -f <yaml file name>** will display like below

```
vagrant@controlplane:~$ kubectldiff -f simple deployment.yaml
diff -u -N /tmp/LIVE-3981462885/apps.v1.Deployment.default.ng-d
--- /tmp/LIVE-3981462885/apps.v1.Deployment.default.ng-dep
+++ /tmp/MERGED-1106384729/apps.v1.Deployment.default.ng-dep
@@ -0,0 +1,41 @@
+apiVersion: apps/v1
+kind: Deployment
+metadata:
+  creationTimestamp: "2026-04-30T21:32:08Z"
+  generation: 1
+  name: ng-dep
+  namespace: default
```

[cropped]

⌘ Run the deployment using: **kubectldapply -f <yaml file name>**

⌘ **kubectldapply view-last-applied deployment <deployment name>**

```
vagrant@controlplane:~$ kubectldapply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  name: ng-dep
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.16.1
        name: nginx
        ports:
        - containerPort: 80
```

➤ Stage 2: [change image using imperative command]

⌘ **kubectl set image deploy <deployment name> <container name>=<new image>**

kubectl set image deploy ng-dep nginx=nginx:1.14.2

```
vagrant@controlplane:~$ kubectl set image deploy ng-dep nginx=nginx:1.14.2
deployment.apps/ng-dep image updated
```

⌘ **kubectl diff -f <deployment file name>**

```
vagrant@controlplane:~$ kubectl diff -f simple deployment.yaml
diff -u -N /tmp/LIVE-2986659376/apps.v1.Deployment.default.ng-dep
--- /tmp/LIVE-2986659376/apps.v1.Deployment.default.ng-dep
+++ /tmp/MERGED-1145231250/apps.v1.Deployment.default.ng-dep
@@ -6,7 +6,7 @@
     kubectl.kubernetes.io/last-applied-configuration: |
       {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"name":"ng-dep","namespace":"default","resourceVersion":"33813"},
       "spec":{"selector":{"matchLabels":{"app":"nginx"},"template":{"metadata":{"labels":{"app":"nginx"},"spec":{"containers":[{"image":"nginx:1.14.2"}]}}}}}}
- generation: 2
+ generation: 3
-   image: nginx:1.14.2
+   image: nginx:1.16.1
     imagePullPolicy: IfNotPresent
```

[you can see, yaml file has *nginx:1.16.1* but live config has *nginx:1.14.2*]

[so, its showing **-** on *nginx:1.14.2* and **+** on *nginx:1.16.1*]

⌘ **kubectl apply view-last-applied deploy <deployment name>**

```
vagrant@controlplane:~$ kubectl apply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  name: ng-dep
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.16.1
        name: nginx
        ports:
        - containerPort: 80
```

[its showing image as *nginx:1.16.1* because imperative change doesn't effect last applied config]

- Stage-3: [update yaml, remove replica field, then declarative update]

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ng-dep
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.16.1
        ports:
        - containerPort: 80
```

```
vagrant@controlplane:~$ kubectl diff -f simple_deployment.yaml
diff -u -N /tmp/LIVE-3508972230/apps.v1.Deployment.default.ng-dep
--- /tmp/LIVE-3508972230/apps.v1.Deployment.default.ng-dep
+++ /tmp/MERGED-2827279498/apps.v1.Deployment.default.ng-dep
@@ -6,14 +6,14 @@
   kubernetes.io/last-applied-configuration: |
     {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"name":"ng-dep","namespace":"default","resourceVersion":"33813","uid":"92189358-0dd5-4f8f-8144-1936af5322ab"},"spec":{"progressDeadlineSeconds":600,"replicas":3,"revisionHistoryLimit":10,"selector":{"matchLabels":{"app":"nginx"},"template":{"metadata":{"labels":{"app":"nginx"},"spec":{"containers":[{"image":"nginx:1.14.2","name":"nginx","ports":[{"containerPort":80}]}]}}}}}
-   generation: 2
+   generation: 3
   name: ng-dep
   namespace: default
   resourceVersion: "33813"
   uid: 92189358-0dd5-4f8f-8144-1936af5322ab
 spec:
   progressDeadlineSeconds: 600
-   replicas: 3
+   replicas: 1
   revisionHistoryLimit: 10
   selector:
     matchLabels:
       app: nginx
   spec:
     containers:
-     - image: nginx:1.14.2
+     - image: nginx:1.16.1
```

[its showing if I update, *replica* and *image* will be updated]

- Now in last applied config, *replicas* field would have been removed because we deleted that field in yaml.

```
vagrant@controlplane:~$ kubectl apply view-last-applied -f simple_deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  name: ng-dep
  namespace: default
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.16.1
        name: nginx
        ports:
        - containerPort: 80
```

[replicas is not there, image is *nginx:1.16.1*]

^ *live config*

```
spec:
  progressDeadlineSeconds: 600
  replicas: 1
```

```
spec:
  containers:
  - image: nginx:1.16.1
    imagePullPolicy: IfNotPresent
```

[the image and replicas got changed]

➤ Its looking like, why there is need of *last applied config* because when we apply, the live config is becoming same as the yaml.

➤ Stage-1: [create declarative from yaml]

^ See the yaml now

```
vagrant@controlplane:~$ cat simple_deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ng-dep
  labels:
    app: ng-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.16.1
        ports:
        - containerPort: 80
```

[added one label only]

```
vagrant@controlplane:~$ kubectl apply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  labels:
    app: ng-deployment
  name: ng-dep
  namespace: default
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.16.1
        name: nginx
        ports:
        - containerPort: 80
```

[last applied config will be same as yaml because of declarative change]

- Stage-2: [add one more label in imperative way: *kubectl edit* command]

```
labels:
  app: ng-deployment
  keyx: valx
```

[live config changed]

```
vagrant@controlplane:~$ kubectl apply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  labels:
    app: ng-deployment
  name: ng-dep
```

[last applied config will be same because it was imperative update]

- Stage-3: [update the replicas using declarative]

```
vagrant@controlplane:~$ cat simple_deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ng-dep
  labels:
    app: ng-deployment
spec:
  replicas: 3
  selector:
```

[updated yaml

file]

```

creationTimestamp: "2026-04-16T16:07:16Z"
- generation: 1
+ generation: 2
  labels:
    app: ng-deployment
    keyx: valx
@@ -16,7 +16,7 @@
  uid: 5071163c-87e2-49db-a1a0-000000000000
  spec:
    progressDeadlineSeconds: 600
-   replicas: 1
+   replicas: 3
  revisionHistoryLimit: 10

```

⚡ [ignore generation]

[`kubectl diff -f <deployment file name>`]

you can see, its not showing *keys: valx* label to be deleted when it is not there in our current yaml file.

⚡ `kubectl apply -f <deployment file name>`

```

vagrant@controlplane:~$ kubectl apply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  labels:
    app: ng-deployment
    name: ng-dep
    namespace: default
spec:
  replicas: 3

```

⚡

[last applied config]

```

creationTimestamp: "2026-04-16T16:07:16Z"
generation: 2
labels:
  app: ng-deployment
  keyx: valx
name: ng-dep
namespace: default
resourceVersion: "37619"
uid: 5071163c-87e2-49db-a1a0-000000000000
spec:
  progressDeadlineSeconds: 600
  replicas: 3

```

⚡

[live config, replicas got increased but that label (keyx: valx) is still there]

➤ So, how does *kubectl apply* work then?

⚡ It does the below

Sets fields, that appear in the configuration (yaml) file, in the live configuration.

Clears fields, that removed from the configuration (yal) file, in the live configuration.

- ⌘ It does the above after comparing with *last applied field*.
- ⌘ If any field is there in [either of *yaml* or *last applied config*] (or) [the field is present in both *yaml* and *last applied config*] then that field will be updated in the *live config*.
- ⌘ In the previous example. The label (keyx: valx) was not there in both *yaml* and *last applied config*, so it didn't track that.

➤ In Simple Words it does the below

It tracks the configs which are present in either *yaml* or *last applied config* and updates the *live config* accordingly. If something is there in *live config* but not there in both *yaml* and *last applied config* then it'll skip that.

Merging changes to primitive fields

Primitive fields are replaced or cleared.

Note: - is used for "not applicable" because the value is not used.

Field in object configuration file	Field in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Set live to configuration file value.
Yes	No	-	Set live to local configuration.
No	-	Yes	Clear from live configuration.
No	-	No	Do nothing. Keep live value.

Merging changes to map fields

Fields that represent maps are merged by comparing each of the subfields or elements of the map:

Note: - is used for "not applicable" because the value is not used.

Key in object configuration file	Key in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Compare sub fields values.
Yes	No	-	Set live to local configuration.
No	-	Yes	Delete from live configuration.
No	-	No	Do nothing. Keep live value.

yaml	last applied config	live config	RES
Y	Y	Y/N	yaml : Y last : update to yaml live : update to yaml
Y	N	Y/N	yaml : Y last : update to yaml live : update to yaml
N	Y	Y/N	yaml : N last : remove live : remove
N	N	Y	yaml : N last : N live : no update

Namespaces

- Consider the below analogy



- ⌘ There are 2 houses,
 - In one house (house-A), one person called *Mark Smith* is there.
 - In another house (house-B), one person called *Mark Williams* is there.
- ⌘ In house-A, people can call *Mark Smith* as only *Mark* because there is no other guy having this name in that house.
 - Similarly in house-B, people can call *Mark Williams* as *Mark* only.
- ⌘ But, if people of house-A wants to call *Mark Williams* then they need to call by full-name i.e. *Mark Williams*. Not only *Mark*.
 - Similarly for people of house-B as well. They need to call *Mark Smith*.
- ⌘ A person who neither belong to house-A nor house-B, needs to call those by their full name only i.e. *Mark Smith* or *Mark Williams*.
- ⌘ Consider these **houses as namespaces**.
 - Inside a namespace, resources having **same names cannot be present** (you can't create 2 pods having exactly same name). but, in multiple namespaces, resources of **same name can be present**.
- ⌘ For example, if you want multiple environments like *dev*, *prod* and you don't want to update any *production* pods while testing during *development* or something like that, **namespaces** help in that.
 - You can have exact same resources in both the namespaces.

- Example:

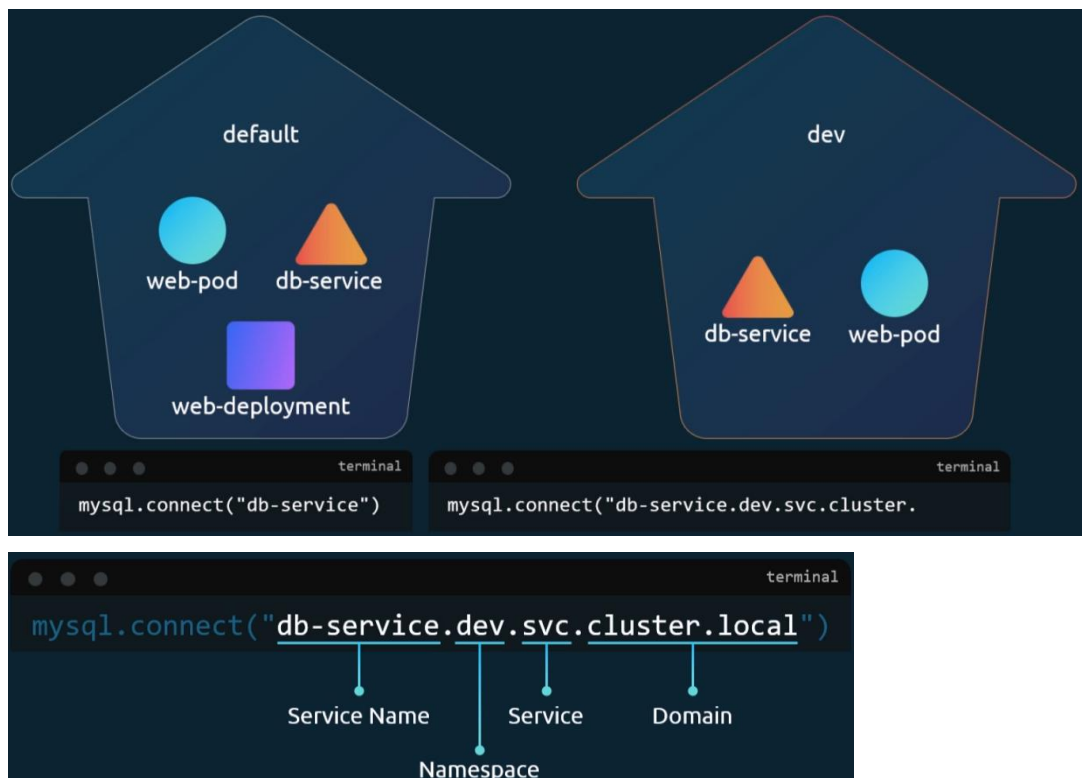
- ⌘ Consider a single namespace scenario:
 - ⌘ Some pods are running DB. And service name (ClusterIP) is **db**.
 - ⌘ The application pods who want to use database can use **db** directly to access the database.
- ⌘ Consider a multiple namespace scenario:

- ✧ In the same example as above, let's imagine 2 namespaces are there.
- ✧ Now, the application pods of one namespace can directly use **db** to access the database running in same namespace.

But, if they want to use **db** of the other namespace then they need to give the full DNS path i.e. **db.<namespace name>**
 or **db.<namespace name>.svc.cluster.local** [you don't need to give this, **db.<namespace name>** is converted to this internally]

✧ db.<namespace name>.svc.cluster.local

in here, **svc** means don't access the pod directly, rather do using the **db** service.
cluster.local is the internal DNS of the cluster. [.local is a **top-level-domain** just like .com, .in ..etc but it is internal]



- ✧ If you want to get or create the resources present in a specific namespace then

kubectl get <resource type like pod, rs, deploy> -n <namespace name>

kubectl create -f <yaml file name> -n <namespace name>

[**NOTE:** You need to write '**create**', not '**apply**' when you are creating something from the yaml **first time** (**still have to verify this**)]

[if the **namespace** key is present in the **yaml** file already, then no need to give **--namespace** flag]

- Or, in *yaml* you can write *namespace* inside metadata section.

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  namespace: dev
```

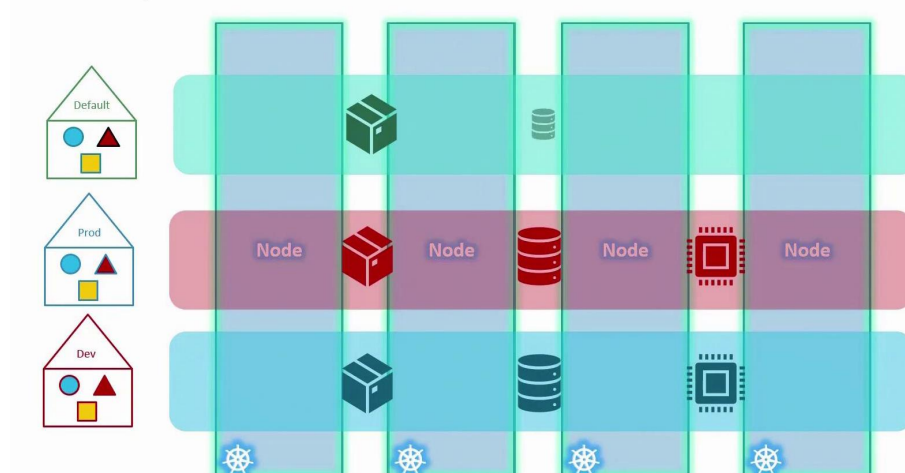
- To create a *namespace*, the *yaml* file format will be like this

```
vagrant@controlplane:~$ kubectl create namespace test --dry-run=client -o yaml
apiVersion: v1
kind: Namespace
metadata:
  name: test
```

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
```

- All the *namespace* configures while setup:
 - When a kubernetes *cluster* is first setup, *default* namespace is created. Whatever resources you create, it is created inside this.
 - Kubernetes creates a set of Pods and Services for its internal purpose. These all stays in the namespace called *kube-system*.
 - There is another namespace called *kube-public* where resources available to all users are created.
- You can create *quota* (long format: *ResourceQuota*) to the namespaces (setting the limits)

Namespace – Resource Limits



- If you want to change the default (not talking about the namespace having name *default*) namespace

```
kubectl config set-context $(kubectl config current-context) --namespace=<namespac  
name which has to be made default>
```

- config -

- The config file path is `~/.kube/config`.

```
apiVersion: v1
clusters:
- cluster:
  certificate-authority-data: LS0tLS1CRUdJTiB...
  server: https://192.168.56.11:6443
  name: kubernetes
contexts:
- context:
  cluster: kubernetes
  namespace: test
  user: kubernetes-admin
  name: kubernetes-admin@kubernetes
current-context: kubernetes-admin@kubernetes
kind: Config
users:
- name: kubernetes-admin
  user:
    client-certificate-data: LS0tLS1CRUdJTiB...
    client-key-data: LS0tLS1CRUdJTiBSU0EgUFJ...
```

-

```
contexts:
- context:
  cluster: kubernetes
  user: kubernetes-admin
  name: kubernetes-admin@kubernetes
current-context: kubernetes-admin@k
```

[if namespace is not there]

it looks something like this (its content format is of yaml, but the filename is **config** only, not **config.yaml**)

- It mainly contains three things: **cluster**, **contexts**, **users**

- **clusters**

- ⌘ its a list of clusters.
- ⌘ Each cluster contains *name*, *cluster* (cluster contains *certificate-authority-data*, *server*) fields.
- ⌘ **name**
 - ⌘ its just an identifier that will be used later inside **context** (you can see inside *context* field).
 - ⌘ every operation i.e. create pod, get svc ..etc **hits this URL**.
kubectl → API Server → Cluster

cluster.server

it's the end-point of *API Server* where all *kubectl* command goes.

↳ cluster.certificate-authority-data

- Ensures you're talking to the real cluster
- Prevents man-in-the-middle attacks

 users

- ♣ It contains the list of users.
- ♣ It defines “Who are you when talking to the Cluster”
- ♣ Each entries of *user* contain the below fields **name**, **user** (*user* contains *client-certificate-data*, *client-key-data*) fields.
- ♣ **name**
just a identifier to the user. Which is referenced inside *context* field (you can see in the yaml).
- ♣ **user.client-certificate-data & user.client-key-data** (common in local setup)
certificate is like ID card; *key* is like signature (Analogy)
- ♣ In real-world, instead of certificate and key, *token* is used.

token: eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXLTJ5In0=
- ♣ it is used in ServiceAccounts, CI/CD, API access.
- ♣ In cloud provider cases, external Auth is present.
- ♣ All-In-One, *user* field is for authentication with the *cluster* .

► **contexts**

- It is something that reference all the things i.e.
`context = cluster + user + default namespace` **--- IMPORTANT ---**
 - When you switch to a context, all the things will be switched i.e. user, cluster, default namespace
 - When you execute any command like
kubecttl get pod
it is executed like the below
kubecttl get pod --namespace=<default namespace name>
 - If the namespace field is not present in the *config* file (refer the image), then it'll take the namespace called **"default"** by default.
 - But, whatever namespace you give (override) like the below
kubecttl get pod --namespace=<namespace name other than context default one>
the **user and cluster will be same only.**
- NOTE** namespace field will not be there inside the *config* file. **-----**

- You can create as many context you want.
 - ⌘ **Same cluster + different namespaces** : new context
 (optional, if you want to maintain different contexts; otherwise you can just override the default namespace and move on in the same context using `--namespace=<other namespace name>`)
 - ⌘ **Same cluster + different user** : new context
 (its mandatory)
 you cannot switch user independently, only you can switch *context*.
 - ⌘ **Different cluster** : new context
 (its mandatory)

➤ **kubectl config --help**

```
Available Commands:
current-context  Display the current-context
delete-cluster   Delete the specified cluster from the kubeconfig
delete-context   Delete the specified context from the kubeconfig
delete-user      Delete the specified user from the kubeconfig
get-clusters     Display clusters defined in the kubeconfig
get-contexts     Describe one or many contexts
get-users        Display users defined in the kubeconfig
rename-context   Rename a context from the kubeconfig file
set              Set an individual value in a kubeconfig file
set-cluster      Set a cluster entry in kubeconfig
set-context      Set a context entry in kubeconfig
set-credentials  Set a user entry in kubeconfig
unset            Unset an individual value in a kubeconfig file
use-context      Set the current-context in a kubeconfig file
view             Display merged kubeconfig settings or a specified kubeconfig file
```

- ⌘ **use-context** is used to set a new **current-context** (see in yaml).
- If you want to create a new context
kubectl config set-context test-context --cluster=kubernetes --user=kubernetes-admin --namespace=test

```
vagrant@controlplane:~$ kubectl config set-context test-context --cluster=kubernetes --user=kubernetes-admin --namespace=test
Context "test-context" created.
vagrant@controlplane:~$ kubectl config get-contexts
CURRENT  NAME                CLUSTER  AUTHINFO  NAMESPACE
*         kubernetes-admin@kubernetes  kubernetes  kubernetes-admin  test
          test-context          kubernetes  kubernetes-admin  test
```

- You can use this command to view all the configs.

kubectl config view

```
vagrant@controlplane:~$ kubectl config view
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: https://192.168.56.11:6443
    name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: kubernetes-admin
    name: kubernetes-admin@kubernetes
current-context: kubernetes-admin@kubernetes
```

[cropped, it'll display the whole config file]

- If you want to change the *current-context* then

kubectl config use-context <context name>

----- THE END OF *CONFIG* -----

- There is another type of resource called **ResourceQuota** which is used to set the **limit** of a specific **namespace**.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: dev
spec:
  hard:
    pods: "10"
    requests.cpu: "4"
    requests.memory: 5Gi
    limits.cpu: "10"
    limits.memory: 10Gi
```

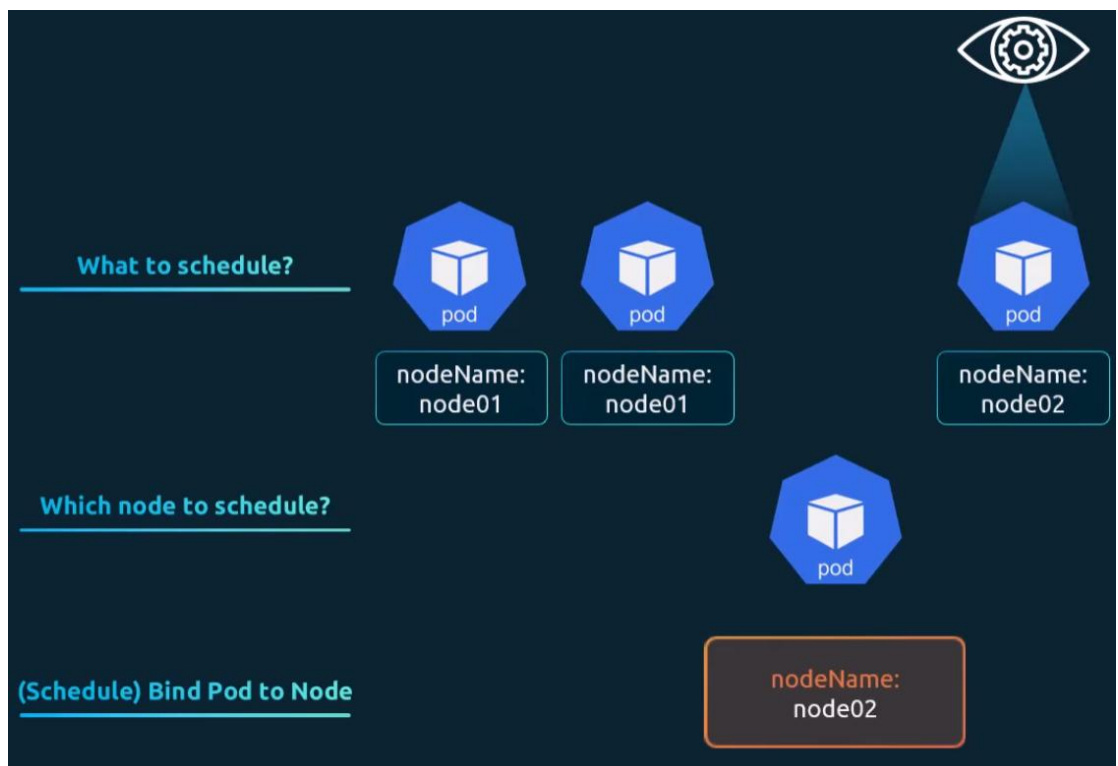


- Scheduling -

- In a running pod, when you see it, there will be a field called **nodeName** will be present, which contains the worker node's name where that specific pod is running. Just execute `kubectrl edit pod <pod name>` to see this.

```
enableServiceLinks: true
nodeName: node02
preemptionPolicy: PreemptLowerPriority
```

- How **scheduling** works?



- ⌘ When you run a pod (without specifying *nodeName* field), the pod object is created.
- ⌘ Then, *scheduler* sees this, and assign a node to it (*nodeName*).
NOTE: this **nodeName** field is under **spec** field of *Pod*'s config.
- ⌘ Then, one **binding** will be created in the target node which will bind the *pod* to the *target node*.
- If the Scheduler doesn't exist,
 - ⌘ then *nodeName* will not be appended to the pod's config.
Even if you give *nodeName*, **binding** object will not be created in the target node unless manually send the POST request to the target worker node to create Binding object.

- ⌘ In this case, the Pod will be in pending state.

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: test-pod
  name: test-pod
spec:
  containers:
  - image: nginx
    name: test-pod
  nodeName: node03
```

Every 2.0s: kubectl get pod

NAME	READY	STATUS	RESTARTS	AGE
ng-dep-66b9cb6d4d-4r822	1/1	Running	0	38h
ng-dep-66b9cb6d4d-f4gl4	1/1	Running	0	38h
ng-dep-66b9cb6d4d-f52lt	1/1	Running	0	39h
test-pod	0/1	Pending	0	13s

- ⌘ I simulated is giving a *nodeName* which doesn't exist inside */etc/hosts* file.
So, the *Binding* object will not be created.

- ⌘ After sometime, the pod will be **deleted** because its status couldn't be 'Running'.

➤ Simulating Manual Scheduling of a Pod to a Node

- ⌘ First create a yaml config of the node giving *nodeName* field.

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: test-pod
  name: test-pod
spec:
  containers:
  - image: nginx
    name: test-pod
  nodeName: node01
```

- ⌘ Now, create the pod

```
vagrant@controlplane:~$ kubectl create -f test-pod.yaml
pod/test-pod created
```

- ⌘ Now, you need to create a *Binding* yaml file.

```
apiVersion: v1
kind: Binding
metadata:
  name: test-binding
target:
  apiVersion: v1
  kind: Node
  name: node01
```

NOTE: It doesn't contain any data about the Pod, it only contains the details of the target node.

- ⌘ The yaml is just to understand the key value pairs, in real there is **no such long-lived object called *Binding* exists.**

Its just the JSON payload to be sent in the POST request to bind the pod to the target node.

- ⌘ Now, you need to trigger the POST request to the **API Server** to bind the pod to the *target worker node*.

NOTE: the worker node doesn't expose any end-points, only API Server in the master node does this.

And, you need to trigger the request to this API Server's end point.

- ⌘ The POST request looks like this (just an example)

```
curl --header "Content-Type: application/json" \
  --request POST \
  --data '{"apiVersion":"v1", "kind": "Binding", "metadata": {"name": "test-binding", "target": {"apiVersion": "v1", "kind": "Node", "name": "node01"}}}' \
  http://$SERVER/api/v1/namespaces/default/pods/$PODNAME/binding
```

- ⌘ You'll get the \$SERVER from **kubectl config view** command. It'll be in the form

<API server node's IP>:<API Server's port>

```
vagrant@controlplane:~$ kubectl config view
apiVersion: v1
clusters:
- cluster:
  certificate-authority-data: DATA+OMITTED
  server: https://192.168.56.11:6443
```

- ⌘ \$PODNAME is the Pod's name that you get from **kubectl get pod** command.

```
vagrant@controlplane:~$ kubectl get pod
NAME                                READY   STATUS
ng-dep-66b9cb6d4d-4r822             1/1     Runni
ng-dep-66b9cb6d4d-f4gl4             1/1     Runni
ng-dep-66b9cb6d4d-f52lt             1/1     Runni
test-pod                             1/1     Runni
```

- ⌘ Now, trigger the request

```
curl --header "Content-Type: application/json" \
  --request POST \
  --data '{"apiVersion":"v1", "kind": "Binding", "metadata": {"name": "test-binding"}, "target": {"apiVersion": "v1", "kind": "Node", "name": "node01"}}' \
  https://192.168.56.11:6443/api/v1/namespaces/default/pods/test-pod/binding/
```

- ⌘ It might not work, because you need to provide token or certificates keys in the URL. **WILL SEE LATER.**

- Labels and Selectors -

- **labels and selectors** are used for **grouping** and **filtering**.
- You can give multiple properties inside **labels** which will be helpful in filtering the resources properly.
- ⌘ You can give as many key value pairs inside **labels** field.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ng-dep
  labels:
    name: ng-deployment
    key1: val1
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
        key2: val2
    spec:
      containers:
```

- ⌘ (here I have given 2 labels to each deployment and pod template)

- ⌘ Then you can select the resources with

--selector key1=val1,key2=val2 (you can give any number of label here)

```
vagrant@controlplane:~$ kubectl get all --selector app=nginx,key2=val2
NAME                                READY   STATUS    RESTARTS   AGE
pod/ng-dep-797565664d-dpkjk         1/1     Running   0           80s
pod/ng-dep-797565664d-qljts         1/1     Running   0           80s
pod/ng-dep-797565664d-zgq4k         1/1     Running   0           80s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/ng-dep-797565664d    3         3         3       80s
```

NOTE: ReplicaSet created by Deployment will have same label as Pods.

- In case of Deployment or ReplicaSet, whatever you write inside **matchLabels** field will be used to get the Pods.
- ⌘ All the fields inside **matchLabels** should be there inside the Pod's labels.
- ⌘ Pod's labels can contain either **same or extra** fields than **matchLabels**, but **not less**.
If pod doesn't contain all the fields present in **matchLabels** then it'll **not be selected**.

➤ Annotations

- ⌘ You can also give a field **annotations** inside *metadata* field.
- ⌘ Annotations may include *build version, contact information, or other integration data*.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: simple-webapp
  labels:
    app: App1
    function: Front-end
  annotations:
    buildversion: "1.34"
```

- ⌘ Annotations are ideal for *adding tool details, version information, or contact data* without impacting how the system groups or selects objects



--- Taints and Tolerations ---

- Real-life analogy (Assumptions HIT kills cockroaches, but cannot kill Lizards):
 - ⌘ You want to prevent cockroaches on your bed. So you sprayed HIT around it.
 - ⌘ Now, Cockroaches cannot come because they are *intolerant* to HIT.
 - ⌘ But, Lizards might come because HIT cannot kill them.
Or, in other words, lizards are *tolerant* to HIT.
- So, How these concepts being used?
 - ⌘ You put *taint* (HIT in the analogy) on the *nodes*, to prevent the *pods* being scheduled to it.
 - ⌘ Only the *tolerant* pods (lizard in the analogy) can be scheduled to the *tainted node*.
 - ⌘ *Nodes* are *tainted* to prevent the *Pods* to be scheduled to them.
Pods are being *tolerated* (when required) to tolerate the *tainted nodes*.
 - ⌘ In simple words,
Taints on nodes repel pods that do not have matching tolerations.
Tolerations in pods allow them to overcome the taint and be scheduled on that node.

➤ Taint the Node

kubectl taint nodes <node name> **key=value:**<taint-effect>

[always try to use plural forms i.e. **nodes** ~~not node~~]

Ex: **kubectl taint nodes node01 env=prod:NoSchedule**

- ⌘ to remove the taint of the node, just put a - at the end.

kubectl taint nodes <node name> **key=value:**<taint-effect>-

Ex: **kubectl taint nodes node01 env=prod:NoSchedule-**

- ⌘ Example:

- ⌘ Initially no *taint* was there.

```
vagrant@controlplane:~$ kubectl describe node node01 | grep -i taint
Taints:                <none>
```

- ⌘ taint node01

```
vagrant@controlplane:~$ kubectl taint nodes node01 env=prod:NoSchedule
node/node01 tainted
```

- ⌘ node01 got tainted.

```
vagrant@controlplane:~$ kubectl describe node node01 | grep -i taint
Taints:                env=prod:NoSchedule
```

- ```
vagrant@controlplane:~$ kubectl taint nodes node01 env=prod:NoSchedule-
node/node01 untainted
```

- ```
vagrant@controlplane:~$ kubectl describe node node01 | grep -i taint
Taints:          <none>
```

• You can also remove the taints for *partial change* by giving this

it'll remove the taint of the node where the key is matched.

```
Taints:      app=product:NoSchedule
             env=dev:NoSchedule
```

To tolerate a pod, there is no *imperative* command, also you **cannot** update using ***kubectrl edit pod*** <pod name> command.

you need to configure the yaml.

```
spec:
  tolerations:
  - key: "env"
    operator: "Equal"
    value: "prod"
    effect: "NoSchedule"
```

[**double-quote** is mandatory]

- Operator: “Equal”
because while giving taint, we wrote *env=prod*

It is because master node has a *taint*

```
Taints:      node-role.kubernetes.io/control-plane:NoSchedule
```

and pods don't have *tolation* to this *taint*.

NoSchedule (we used this previously)

- New Pods will **not be scheduled** on this node *if they are **not having the toleration.***

⌘ Existing Pods will remain as it is. (*even if they doesn't have the toleration*)

⌘ In simple terms:

New Pods: ❌

Existing Pods: ✅

⌘ In our analogy, you put HIT around of the bed. Cockroaches who are already on the bed will survive but no outside cockroaches can come.

➤ **PreferNoSchedule**

⌘ Kubernetes will try to avoid this node to schedule the pods.

⌘ But, it might schedule if there is no other option.

⌘ In simple terms:

New Pods: ❌✅

Existing Pods: ✅

➤ **NoExecute**

⌘ New Pods will not be scheduled to this node (same as *NoSchedule* effect)

⌘ Existing Pods will be removed (different from *NoSchedule* effect)

⌘ In simple terms:

New Pods: ❌

Existing Pods: ❌

⌘ In our analogy, you put HIT around the bed and also on the bed. Now cockroaches will be dead and no outside cockroaches can come. Lizards are safe.

➤ **NOTE:** All the not-scheduling and delete from the node things which are written above, those are for the pods who don't have *tolerance* to that *taint*.

If the pods are having *tolerance*, then they (new and existing) will not be affected.

⌘ -----

--- *nodeSelector and nodeAffinity* ---

- While *taint* and *tolerations* were to avoid a node to be scheduled for pods, *nodeSelector* and *nodeAffinity* are there to schedule a node for the pods.

➤ *nodeSelector* -----

- It works in the same principle *selector*.
- First provide the *labels to the node*. [before we labelling the Pods]

```
kubectl label nodes node1 env=prod
```

- Use *nodeSelector* field to match the Node labels. [ReplicaSet's *selectors* field]

```
spec:
  nodeSelector:
    env: prod
```

- Example

- ↪ Created a 2 pod yaml files (one for env=prod, another for env=dev)

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    app: prod-pod
    name: prod-pod
spec:
  nodeSelector:
    env: prod
  containers:
  - image: nginx
    name: prod-cont
    ports:
    - containerPort: 80
      name: nginx-port
```

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    app: dev-pod
    name: dev-pod
spec:
  nodeSelector:
    env: dev
  containers:
  - image: nginx
    name: dev-cont
    ports:
    - containerPort: 80
      name: nginx-port
```

- ↪ Label *node01* as *env=dev*, and *node02* as *env=prod*

```
vagrant@controlplane:~$ kubectl label nodes node01 env=dev
node/node01 labeled
vagrant@controlplane:~$ kubectl label nodes node02 env=prod
node/node02 labeled
```

```
Labels:          beta.kubernetes.io/arch=amd64
                 beta.kubernetes.io/os=linux
                 env=dev
```

[node01]

```
Labels:          beta.kubernetes.io/arch=amd64
                 beta.kubernetes.io/os=linux
                 env=prod
```

[node02]

- Created prod pod and it was scheduled in node02

```
vagrant@controlplane:~$ kubectl create -f prod-pod.yaml
pod/prod-pod created
vagrant@controlplane:~$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
prod-pod	1/1	Running	0	8s	10.244.140.103	node02	<none>		<none>	

- Created dev pod and it was scheduled in node01

```
vagrant@controlplane:~$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
dev-pod	1/1	Running	0	36s	10.244.196.162	node01	<none>		<none>	

➤ nodeAffinity -----

- The *nodeSelector* was very straight forward. You need to give *label* properly [hard selection just like *matchLabels* in ReplicaSet]
nodeAffinity is having multiple types of label mathing. [like *matchExpressions* in ReplicaSet]

- What if you want to say like
 place the pod *where env=prod or env=test*
 place the pod *where env != dev*
 in this case, ***nodeAffinity*** can be used.

- The yaml looks like the below

```
spec:
  containers:
    - name: data-processor
      image: data-processor
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: size
                operator: In
                values:
                  - Large
```

```
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 5
          preference:
            matchExpressions:
              - key: zone
                operator: In
                values: [us-east-1]
```


- 2 types of nodeAffinity

requiredDuringSchedulingIgnoredDuringExecution

preferredDuringSchedulingIgnoredDuringExecution

- Pods are having 2 stage: *scheduling, execution*

- ⌘ When the pod is not created in any node, it is first scheduled. [**Scheduling**]

- ⌘ After the pod being scheduled, it'll be created in the scheduled node. [**Execution**]

- ⌘ Currently, nodeAffinity is only there for *scheduling* phase, you can see its written **Ignored during Execution.**

- *required* one contains ***nodeSelectorTerms*** whereas *preferred* one contains *weight* and *preference* .

- In case of **requiredDuringSchedulingIgnoredDuringExecution**, if the expression doesn't match with any node, then the **Pod will not be scheduled at all.** [**HARD** matching]
In case of **preferredDuringSchedulingIgnoredDuringExecution**, all the nodes will be checked and weight will be calculated. **Highest preferred node will be scheduled for the pod.** [**SOFT** matching]

- How the *weight* is calculated in case of **preferredDuringSchedulingIgnoredDuringExecution** ?

- ⌘ If you have provided some preferences, all will be checked in all the nodes available.

Each preference will be having some weight (you'll define this)

- ⌘ The nodes will be checked for all the preferences. If preference matches in that node then weight for that node will be increased with the weight provided for that preference.

```
preferredDuringSchedulingIgnoredDuringExecution:
- weight: 5
  preference:
    matchExpressions:
      - key: env
        operator: In
        values: [prod]

- weight: 3
  preference:
    matchExpressions:
      - key: disk
        operator: In
        values: [ssd]
```

Node	env=prod	disk=ssd	Score
node1	✓	✓	5 + 3 = 8
node2	✓	✗	5 + 0 = 5
node3	✗	✓	0 + 3 = 3

here, *node1* has both the preference matched so its weight will be 8, so pods will be scheduled to *node1*.

- Use `kubectl explain pod.spec.affinity.nodeAffinity --recursive` to get the fields and directly paste inside pod yaml file.
- Here we used *matchExpressions* to match the label of the nodes, there is another called *matchFields*.
 - ♣ It matches any type of fields you want, not just *labels*.

```
matchFields:
- key: metadata.name
  operator: In
  values: [node1]
```

[metadata.name must be node1]

- There are other 2 types of affinity, **podAffinity**, **podAntiAffinity**
 - ♣ *nodeAffinity* checks the labels in the nodes and match that.
 - ♣ *podAffinity* and *podAntiAffinity* checks the labels of the pods which are running inside the node.
- **podAffinity**
 - ♣ The pod running on target nodes will be checked with the preferences/nodeSelectorTerms. if matches then that node will be scheduled.
- **podAntiAffinity**
 - ♣ Opposite of *podAffinity*, the lowest matched (in case of *preferredDuring...*) will be selected. or not matched (in case of *requiredDuring...*) will be selected.
- Some operators of *matchExpressions*

```
- matchExpressions:
  - key: size
    operator: In
    values:
      - Large
```

[In]

```
- matchExpressions:
  - key: size
    operator: Exists
```

[Exists]

^ All the operators are

[kubectl explain](#)

[pod.spec.affinity.nodeAffinity.preferredDuringSchedulingIgnoredDuringExecution.preference.matchExpressions](#)

```
Possible enum values:
- `DoesNotExist`
- `Exists`
- `Gt`
- `In`
- `Lt`
- `NotIn`
```

➤ F

--- Resource Requirements ---

- There are 2 things, **Request** and **Limit**.
 - ⌘ Request: minimum required resource (used for scheduling)
 - ⌘ Limit: maximum allowed resource (enforced at runtime)
- The YAML look like this.

```
apiVersion: v1
kind: Pod
metadata:
  name: demo
spec:
  containers:
  - name: app
    image: nginx
    resources:
      requests:
        cpu: "500m"
        memory: "256Mi"
      limits:
        cpu: "1"
        memory: "512Mi"
```

[**resources** is written **inside containers**, not **spec**] < -----

- **requests** (Scheduling Decision)
 - ⌘ If any node has at least this much free resources, then only that node can be scheduled for the pod.
Otherwise it'll not be scheduled.
- **limits** (Runtime enforcement)
 - ⌘ CPU: throttled (if exceed than the limit set) [throttled means slow down]
 - ⌘ Memory: container is **killed (OOMKilled)** (if exceed than the limit set)
- CPU vs Memory behaviour
 - ⌘ CPU can exceed request, but Memory cannot.
 - ⌘ In case of hitting the limit,
CPU: throttled [slow down]
Memory: *container killed*

- Resource is not there, then by default it takes as 0.
- But, if resource is not there and limit is there: resource will be taken same as limit.
- If limit is not there, then it'll assume the pod can use *unlimited* resources.
- 1 (one) count of CPU is same as
 - ♣ 1 AWS vCPU
 - ♣ 1 GCP Core
 - ♣ 1 Azure Core
 - ♣ 1 Hyperthread
- Memory
 - ♣ 1 G : 1 Gigabyte
 - ♣ 1 M : 1 Megabyte
 - ♣ 1 K : 1 Kilobyte
 - ♣ 1 Gi : 1 Gibibyte
 - ♣ 1 Mi : 1 Mebibyte
 - ♣ 1 Ki : 1 Kibibyte

[K : 1000 bytes, Ki: 1024 bytes ; K is binary (2's power), Ki is decimal (10's power)]
- We can have 4 scenarios:
 - No requests, No limits
 - No requests, Limits
 - Requests, Limits
 - Requests, No limits
 - ♣ In the first case, if one pod (container inside pod because requests and limits are set per container, not per pod) is consuming all the CPU, and another pod is also assigned to same node (because if request is not given, it'll be **taken as 0**) which is not able to get the resources.
 - ♣ In the second case, request will be **taken as same as limits**.
 - ♣ In the 3rd case, the pods are having both request and limits. Lets say in this case pod-1 doesn't need extra resources but pod-2 needs.
And we have set the limit for both the pods which will stop *pod-2* to use more resources. Which is not ideal
 - ♣ In the 4th case, the problem of 3rd case can be solved. Because if a pod is being assigned to a node means the node has required amount of resources available that the pod needs.
And, there is no limit so pod can use as much as it want.

- There is something called **LimitRange**, it by default injects the **requests** and **limits** to the pods if not specified in the yaml.

⌘ **It works in namespace level.**

```
apiVersion: v1
kind: LimitRange
metadata:
  name: cpu-resource-constraint
spec:
  limits:
    - default:
        cpu: 500m
      defaultRequest:
        cpu: 500m
      max:
        cpu: "1"
      min:
        cpu: 100m
      type: Container
```

The diagram illustrates the mapping of fields within the `spec.limits` array to `Limit` and `Request` values. It shows four entries in the `limits` array, each with a `default`, `defaultRequest`, `max`, and `min` field. Lines connect these fields to labels on the right: `default` and `max` are connected to `Limit`, while `defaultRequest` and `min` are connected to `Request`.

⌘ **default :**

- it is the **limits** field.
- If the pod yaml doesn't contain **limits** fields, then this LimitRange's `spec.limits.default` will be added there.

⌘ **defaultRequest :**

- It is the **requests** field.
- If the pod yaml doesn't contain **requests** fields, then this LimitRange's `spec.requests` default will be added there.

⌘ **min**

- The minimum amount of resource a container can ask for (**requests**).
- If the container (inside pod) asks lesser resource then this `min` then it'll be rejected.

⌘ **max :**

- Same as `min`.
- The container cannot request more requests then that defined in `max`.

⌘ **type : Container**

➤

➤ **DaemonSet**

- ⌘ It is similar to ***ReplicaSet***
- ⌘ It makes sure that each node should contain one copy of the pod specified in *DaemonSet* configuration.
- ⌘ Earlier it used to append ***nodeName*** field and POST request to create the container in the target node.
But now, it uses ***nodeAffinity*** to assign a specific node to the Pod.
- ⌘ The yaml is exact same as that of *ReplicaSet*, just the ***kind*** will be different.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: monitoring-daemon
spec:
  selector:
    matchLabels:
      app: monitoring-agent
  template:
    metadata:
      labels:
        app: monitoring-agent
    spec:
      containers:
        - name: monitoring-agent
          image: monitoring-agent
```

⌘

➤ F

- Static Pods -

- Pods are created by the command of *kube-api-server*, which is listened by *kubelet* (in worker nodes) and create and start the container.
what if the worker node is not connected to any Kubernetes cluster, it just has *kubelet* installed. How to run a pod here?
- You need to find the ***staticPodPath*** in the node where *kubelet* is present.
kubelet constantly see this path and create and update the pod if any *yaml* config is present inside this path.
- **NOTE :-** You can only create pods in this way, other resources like *ReplicaSet*, *Deployment* cannot be created because they are completely of *Kubernetes* control plane things.
- To find the ***staticPodPath*** the below methods are there [all in worker node, not master node]

Method-1

✧ `systemctl cat kubelet`

```
# Note: This dropin only works with kubeadm and kubelet v1.11+
[Service]
Environment="KUBELET_KUBECONFIG_ARGS=--bootstrap-kubeconfig=/etc/kubernetes/bootstrap-kubelet.conf --kubeconfig=/etc/
Environment="KUBELET_CONFIG_ARGS=--config=/var/lib/kubelet/config.yaml"
# This is a file that "kubeadm init" and "kubeadm join" generates at runtime, populating the KUBELET_KUBEADM_ARGS v
EnvironmentFile=/var/lib/kubelet/kubeadm-flags.env
# This is a file that the user can use for overrides of the kubelet args as a last resort. Preferably, the user sho
# the .NodeRegistration.KubeletExtraArgs object in the configuration files instead. KUBELET_EXTRA_ARGS should be so
EnvironmentFile=/etc/default/kubelet
ExecStart=
ExecStart=/usr/bin/kubelet $KUBELET_KUBECONFIG_ARGS $KUBELET_CONFIG_ARGS $KUBELET_KUBEADM_ARGS $KUBELET_EXTRA_ARGS
```

(you'll get something like this; this image is cropped)

if you find something like below, then see the `--pod-manifest-path`

```
ExecStart=/usr/local/bin/kubelet \
  --container-runtime=remote \
  --container-runtime-endpoint=unix:///var/run/cont
  --pod-manifest-path=/etc/Kubernetes/manifests \
  --kubeconfig=/var/lib/kubelet/kubeconfig \
  --network-plugin=cni \
  --register-node=true \
  -v=2
```

- ✧ Open that ***config.yaml*** file present there, you'll find a key called

staticPodPath

```
staticPodPath: /etc/kubernetes/manifests
```

- ✧ In this path i.e. ***/etc/kubernetes/manifests***, if you create a Pod's *yaml* file, then *kubelet* will create that pod inside that node.

- The above displayed is `kubelet.service` file (it is same as the above `systemctl cat kubelet`)
- Its mostly in `/usr/lib/systemd/system/kubelet.service` if not, then find other service file directories like `/etc/systemd/system`, `/var/lib/systemd/system` ..etc
- There you'll find the `ExecStart` field

```
[Unit]
Description=kubelet: The Kubernetes Node Agent
Documentation=https://kubernetes.io/docs/
Wants=network-online.target
After=network-online.target

[Service]
ExecStart=/usr/bin/kubelet
Restart=always
StartLimitInterval=0
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Method-2

- `ps -ef | grep kubelet`

```
vagrant@node01: /usr/local/bin$ ps -ef | grep kubelet
root      3762      1   2 May02 ?           00:32:17 /usr/bin/kubelet --bootstrap-kubeconfig=/etc/kubernetes/bootstrap-kubelet.conf --kubeconfig=/etc/kubernetes/kubelet.conf --config=/var/lib/kubelet/config.yaml --node-ip 192.168.56.21
root      4946      1   2 May02 ?           00:00:05 /usr/bin/pod-driver --podman-socket=/var/lib/containers/containers.sock
```

- You'll find the `config.yaml` path here.
- Go to that `yaml` file and get the `staticPodPath`.

➤ To check in the worker node

- You can use `crictl` command to see if the container is running.
You can't use `kubectl` command there because its not kubernetes.
- `crictl` is used to talk to `containerd`.
If you get "Permission Denied" then run with `sudo`.
- `sudo crictl ps`

CONTAINER	IMAGE	CREATED	STATE	NAME	ATTEMPT	POD ID	POD	NAMESPACE
306e8944e30c3	6c3a6ea6608c8	26 minutes ago	Running	static-cont	0	37b86e808bd54	test-static-pod-node01	default
60713133410e0	6c3a6ea6608c8	16 hours ago	Running	dev-cont	0	66d3a3015d3e0	dev-pod	default

- `kubeadm` creates **static pods** in master node to run the **control plane resources**.

```
vagrant@controlplane: /etc/kubernetes/manifests$ ls
etcd.yaml kube-apiserver.yaml kube-controller-manager.yaml kube-scheduler.yaml
```

- When you execute `kubectl get pods` in the control plane, you'll see the `static pod` there.

HOW? Because this should be specific to the worker node and kube API Server shouldn't know about this. Read about the **end-to-end flow** (next page) for this.

| End-To-End flow from Pod creation command to actual Running of Pod |

----- IMPORTANT -----

- Below is the flow end-to-end
 - ⌘ User creates a Pod
 - ⌘ *kubectl apply -f my-pod.yaml* (or) *kubectl run ...*
 - ⌘ This request goes to API Server.
 - ⌘ API Server
 - ⌘ It validates the request.
 - ⌘ Store the *Pod* object (PodSpec) in *etcd*.
 - ⌘ Scheduler
 - ⌘ It sees the unscheduled pod in *etcd*.
 - ⌘ It schedule any node to this pod [updates *spec.nodeName*]
 - ⌘ Kubelet (in the scheduled node)
 - ⌘ Watches API Server, and see a pod is assigned to it.
[kubelet opens a *watch* connection. API Server streams updates through it]
 - ⌘ Pulls the *PodSpec*.
 - ⌘ Kubelet → Container runtime (*containerd*)
 - ⌘ Creates Pod sandbox (pause container: otherwise container will get its own IP; instead of using Pod's IP)
 - ⌘ Creates all the required containers defined in *PodSpec*.
 - ⌘ Container Runtime (*containerd*)
 - ⌘ Pulls images, & creates the containers.
 - ⌘ Assign *container IDs*.
 - ⌘ Start the processes.
 - ⌘ Kubelet (after creation)
 - ⌘ Maps *Pod ID* → *Container IDs*
 - ⌘ Monitors the containers
running / crashed / restarted
 - ⌘ Kubelet → API Server (status update)
 - ⌘ API server doesn't poll for Pod's status.
Kubelet update API server about the Pod's status.
 - ⌘ API Server stores updated status in *etcd*.

```
status:
  phase: Running
  containerStatuses:
    - name: nginx
      containerID: containerd://abc123
      state: running
```

- ⌘ Control Plane (now updated)
 - ⌘ *etcd* contains
 - PodSpec (desired state)
 - PodStatus (actual state from Kubelet)
- ⌘ Controllers (ReplicaSets, etc..)
 - ⌘ Watch API Server
 - ⌘ Compares
 - Desired state vs Current state
 - ⌘ Take action if mismatch.
- Now, the question arises: Does the ReplicaSet or other controllers repeatedly poll API Server about the pod's status?
 - ⌘ No, it starts a *watch event*.
 - ⌘ Its not a *web-hook*, it's a long-lived connection (*streaming connection*)
 - ⌘ Whenever something changes, API Server streams it, so *controllers* get to know about the pod's status.
- Control Plane knows nothing about the containers.

It gets the details about the containers (id and status) via Kubectl only.

Component	Knows PodSpec	Knows Container ID	Runs Containers
API Server	✓	⚠ (reported only)	✗
etcd	✓	⚠ (stored only)	✗
kubelet	✓	✓ (authoritative)	✗
containerd	✗	✓ (real)	✓

- To see the details about the containers in *Control Plane* and *Worker Node*
 - ⌘ When you execute


```
kubectl get pod <pod name> -o yaml
```

(or)

```
kubectl edit pod <pod name>
```

 - ⌘ It shows the content of *etcd* in yaml format.
 - ⌘ **status.containerStatuses**
 - this field contains the list of containers associated with that pod along with their

IDs, image, state (running, etc.), volumes ..ec.

```
containerStatuses:
- containerID: containerd://98243133419e0d62
  image: docker.io/library/nginx:1.29
  imageID: docker.io/library/nginx@sha256:6
  lastState: {}
  name: dev-cont
  ready: true
  resources: {}
  restartCount: 0
  started: true
  state:
    running:
      startedAt: "2026-05-02T18:10:45Z"
  user:
    linux:
      gid: 0
      supplementalGroups:
        - 0
      uid: 0
  volumeMounts:
    - mountPath: /var/run/secrets/kubernetes.:
```

➤ NOTE

- ⌘ **kubelet** has a watch connection with API Server.
And, whenever API Server gets some request (either create a new pod or update an existing pod), it updates the *etcd*.
- ⌘ When you trigger **kubectrl edit pod <pod name>**
 - ⌘ API Server gets the update request, it updates the *etcd* if there is any changes.
 - ⌘ **kubelet** already has a *watch connection*, through that API Server sends the modified *PodSpec*.
 - ⌘ **kubelet** gets the new *PodSpec* and compare with the existing one.
 - ⌘ If any changes, then it recreates the container (if allowed; if not allowed then reject it)
it sends the report of new container to the API Server i.e. new **Container ID**, ..etc.

```
containerStatuses:
- containerID: containerd://3d835f86cfac50fe61ea18455f9b7a9a22f98e73aa22d2c34dd54d4c7bf9b76b
  image: docker.io/library/nginx:1.29-perl
  imageID: docker.io/library/nginx@sha256:3d5526f7dc2dfd0b6cbebc664ca2d7ab1e1b5f572a289c6a7607728c5f5b85c
  lastState:
    terminated:
      containerID: containerd://98243133419e0d62baa1134c86c58fa102f93170f3ff7f82e08ebce518078c96
      exitCode: 0
      finishedAt: "2026-05-03T13:08:32Z"
      reason: Completed
      startedAt: "2026-05-02T18:10:45Z"
```

[you can see, lastState.terminated; it contains previous containerID here]

[contains only last terminated container details, not all]

- In case of no change, doesn't do anything.
- In case of **static pod**, *kubelet* sends the same report to API Server; but **with some extra fields**

- The below is the *metadata* of normal pod (pod created by api server)

```
metadata:
  annotations:
    cni.projectcalico.org/containerID: c6d2a3016d3a055cededa48f1ca2c97a5c69303ce534161abd71d26a4e4c2641
    cni.projectcalico.org/podIP: 10.244.196.162/32
    cni.projectcalico.org/podIPs: 10.244.196.162/32
```

- The below is the *metadata* of **static pod** (pod created in worker node itself)

```
metadata:
  annotations:
    cni.projectcalico.org/containerID: 37b86e808bd5439f78bf763fa7914144be4b67de4342957d3253bc2412eaa40a
    cni.projectcalico.org/podIP: 10.244.196.163/32
    cni.projectcalico.org/podIPs: 10.244.196.163/32
    kubernetes.io/config.hash: af6e265c669e39311b5f3774274e0832
    kubernetes.io/config.mirror: af6e265c669e39311b5f3774274e0832
    kubernetes.io/config.seen: "2026-05-03T09:51:34.943426976Z"
    kubernetes.io/config.source: file
```

- You can see 2 key value pairs:

kubernetes.io/config.mirror: <hash>

kubernetes.io/config.source: file

these tells API Server that **this is not a Pod created by API Server; this is a mirror of a static pod.**

- **config.source: file** means the pod is created from a *file* present in the *node* itself;

if this field is not there means the pod is created by API Server itself.

- The API server stores mirror pod objects also in etcd, but they are not the source of truth. They are copies of static pods created by kubelet and marked with annotations to indicate their origin.

----- *Multiple Schedulers* -----

- F
- F
- F
- F
- F
- F
- F
- F
-

Important Points

- Try to use *imperative commands* as much as you can in Exam.
- If you forget the keywords and resources, then execute **kubect! api-resources** you'll get the keywords.

```
vagrant@controlplane:~$ kubectl api-resources
```

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
bindings		v1	true	Binding
componentstatuses	cs	v1	false	ComponentStatus
configmaps	cm	v1	true	ConfigMap
endpoints	ep	v1	true	Endpoints
events	ev	v1	true	Event
limitranges	limits	v1	true	LimitRange
namespaces	ns	v1	false	Namespace
nodes	no	v1	false	Node
persistentvolumeclaims	pvc	v1	true	PersistentVolumeClaim
persistentvolumes	pv	v1	false	PersistentVolume
pods	po	v1	true	Pod
podtemplates		v1	true	PodTemplate
replicationcontrollers	rc	v1	true	ReplicationController
resourcequotas	quota	v1	true	ResourceQuota
secrets		v1	true	Secret
serviceaccounts	sa	v1	true	ServiceAccount
services	svc	v1	true	Service
mutatingadmissionpolicies		admissionregistration.k8s.io/v1	false	MutatingAdmissionPolicy
mutatingadmissionpolicybindings		admissionregistration.k8s.io/v1	false	MutatingAdmissionPolicyBinding
mutatingwebhookconfigurations		admissionregistration.k8s.io/v1	false	MutatingWebhookConfiguration

[you can see there is a column 'KIND' which you can directly write in the yaml]

- ⌘ **Shortnames** are also there to use in *imperative commands* which will SAVE TIME.
- If you want to get a documentation regarding any specific resource, you can use **kubect! explain <resource name>** command.

```
vagrant@controlplane:~$ kubectl explain pods
```

KIND: Pod
VERSION: v1

DESCRIPTION:
Pod is a collection of containers that can run on a host. This resource is created by clients and scheduled onto hosts.

FIELDS:

apiVersion <string>
APIVersion defines the versioned schema of this representation of an object. Servers should convert recognized schemas to the latest internal value, and may reject unrecognized values. More info:
<https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#resources>

kind <string>
Kind is a string value representing the REST resource this object represents. Servers may infer this from the endpoint the client submits requests to. Cannot be updated. In CamelCase. More info:
<https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#types-kinds>

metadata <ObjectMeta>
Standard object's metadata. More info:
<https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#metadata>

spec <PodSpec>
Specification of the desired behavior of the pod. More info:
<https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#spec-and-status>

status <PodStatus>
Most recently observed status of the pod. This data may not be up to date. Populated by the system. Read-only. More info:
<https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#spec-and-status>

- ^ To go deeper into the specific fields of that resources,

kubectl explain <resource name>.<field name>

[you can nest these any number times:

resource.field.sub_field.sub_sub_field etc]

```
vagrant@controlplane:~$ kubectl explain pods.spec
KIND:      Pod
VERSION:   v1

FIELD: spec <PodSpec>

DESCRIPTION:
  Specification of the desired behavior of the pod. More info:
  https://git.k8s.io/community/contributors/devel/sig-architecture/api-convent
  PodSpec is a description of a pod.

FIELDS:
  activeDeadlineSeconds <integer>
    Optional duration in seconds the pod may be active on the node relative to
```

If you

want, more comprehensive, then use

kubectl explain <resource name> --recursive

```
vagrant@controlplane:~$ kubectl explain pods --recursive
KIND:      Pod
VERSION:   v1

DESCRIPTION:
  Pod is a collection of containers that can run on a host
  created by clients and scheduled onto hosts.

FIELDS:
  apiVersion <string>
  kind <string>
  metadata <ObjectMeta>
    annotations <map[string]string>
    creationTimestamp <string>
    deletionGracePeriodSeconds <integer>
```

- To get a basic yaml file instead of writing from scratch, use **--dry-run=client -o yam**l with the command.

[with **run**, **create** for static] [with **get** if the object is already running]

kubectl create deployment my-dep --image=nginx --dry-run=client -o yaml

kubectl run my-pod --image nginx --dry-run=client -o yaml

```
vagrant@controlplane:~$ kubectl run my-pod --image=nginx --dry-run=client -o yam
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: my-pod
    name: my-pod
spec:
  containers:
  - image: nginx
    name: my-pod
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
```

^

```

vagrant@controlplane:~$ kubectl create deployment my-dep --image=nginx --dry-run=client -o yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: my-dep
    name: my-dep
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-dep
  strategy: {}
  template:
    metadata:
      labels:
        app: my-dep
    spec:
      containers:
      - image: nginx
        name: nginx
        resources: {}

```

- **config** file is present in `~/.kube/config` which contains all the details about *users*, *clusters*, *contexts*.
 - ⌘ **cluster** contains the list of *kube-api-server endpoint & certificate* (for identity).
 - ⌘ **users** contains the list of users each having a *unique name, key, certificate*.
 - ⌘ **context** contains the combination of triplet i.e. *user, cluster, namespace*.
[default namespace]
 - ⌘ When you switch a context, the user will be updated, the cluster will be updated, and the default namespace will be updated.
You can override the default namespace with the flag **--namespace=<ns name>**
 - ⌘ [kubectl config --help](#)
command you can use to see the usage of different fields of configs.
- TAINT, TOLERATIONS, NODE SELECTOR, NODE AFFINITY,
 - ⌘ **taint** is to prevent a pod to be scheduled on a node (taint is applied on node).
 - ⌘ **toleration** is to make a pod tolerant to get scheduled to a *taint* pod (toleration is applied on pod).
 - ⌘ **nodeSelector** is used to assign a pod to a node only if the selector labels matches with the node's labels. [HARD selection, if not matches then it is not scheduled].
 - ⌘ **nodeAffinity** is advanced version of *nodeSelector*, here *hard* and *soft* selection can be there with advanced selection queries.
 - ⌘ Sometimes you need to use both the things **taint**, **toleration** and **nodeAffinity** to solve the real-world problem.

- You can give **requests** and **limits** on the pods.
 - ⌘ **requests** means the pod's minimum requirement (cpu and memory) to be scheduled to a node.
 - ⌘ **limits** means the pod can use that much amount of resources at max.
 - ⌘ If **cpu** usage exceeds the **limit**, then it **slows down**.
 - ⌘ If **memory** usage exceeds, then pod is **killed**.
 - ⌘ Instead of giving this **requests** and **limits** in all the pods, you can create **LimitRange** (kind) which will contain the **request, limits, max, min** .
 - ⌘ If any pod doesn't have the request and limit fields, then it'll inject its default values.
 - ⌘ **LimitRange** works at **namespace** level.
- **DaemonSet** is same as **ReplicaSet**, it just create a replica of the pod in each worker node.
- **kubeadm** creates static pods on control plane nodes to run the controlplane resources.
- API Server stores the details of all the pods in **etcd** (**normal pods** or **static pods**).

in the annotation 2 extra fields are there in case of **static pods** which differentiate it from the normal pods.

 - ⌘ When you edit a pod using **kubectrl edit** command, it updates etcd and send **kubelet** (in streaming connection) the new **PodSpec** then **kubelet** compares it with the existing **PodSpec**.

Then changes if allowed otherwise reject.

In each changes, **containerd** creates new container and the container ID is changed.
 - ⌘ In case of **static pods**, you can't change using **kubectrl edit** command in control plane nodes.

You need to update the **yaml** file inside the **manifest** directory of the target node (where the static pod is running).
 - ⌘
- F
- F
- F